Lab Task 3

A software company is developing a User Registration API using NestJS. The application allows new users to register by providing their personal information, account credentials, and contact details. During testing, the development team discovers that users can submit incomplete names, invalid email addresses, weak passwords, negative ages, incorrectly formatted phone numbers, and unsupported account types. You are assigned as a backend developer to implement input validation so only correctly formatted data is accepted. Use DTOs, class-validator decorators, and ValidationPipe to check requests before they are processed by the controller.

Task Requirements

Task 1 — Create a Registration DTO (2 Marks)

Create a CreateUserDto class with the following fields and appropriate validation decorators.

Input Field	Validation Requirement
fullName	Required string; minimum 3 characters
email	Required; valid email address
password	At least 8 characters, including uppercase, lowercase, number, and special character
age	Required integer from 18 to 60
phone	Required string of exactly 11 digits beginning with 01
accountType	Must be either "student" or "teacher"
address	Optional string; if supplied, minimum 10 characters

Task 2 — Configure Global ValidationPipe (1 Mark)

Enable ValidationPipe in main.ts with whitelist: true, forbidNonWhitelisted: true, and transform: true. Unexpected properties must be rejected.

Task 3 — Implement Registration API (1 Mark)

Implement POST /users/register in a NestJS controller. Accept the request body using @Body() and CreateUserDto. For valid data, return a success message along with the user's name, email, age, and account type. Do not return the password.

Task 4 — Test Using Postman (2 Marks)

Demonstrate each request and its response:

Test	Input Condition	Expected Result
1	All fields valid	201 Created
2	Invalid email address	400 Bad Request
3	Weak password	400 Bad Request
4	Age below 18	400 Bad Request
5	Unexpected isAdmin property	400 Bad Request

Submission Instructions

Submit the DTO, Controller, and main.ts validation configuration, plus Postman screenshots showing successful and failed requests. Use NestJS and class-validator. Do not perform manual field validation with if/else inside the controller; use DTO decorators and ValidationPipe.

Marking Rubric

Assessment Criterion	Marks
DTO and validation decorators	2
Global ValidationPipe configuration	1
Controller and POST endpoint	1
Five Postman validation tests	2

Total	6

Sample Requests

Valid request: POST http://localhost:3000/users/register

{"fullName":"Rahim Ahmed","email":"rahim@gmail.com","password":"Rahim@123","age":22,"phone":"01712345678","accountType":"student","address":"Dhaka, Bangladesh"}

Invalid example: {"fullName":"Ra","email":"invalid-email","password":"12345","age":15,"phone":"1234","accountType":"admin"}

Instructor note: If using numeric transformation in DTOs, students may use @Type(() => Number) with integer and range validators. Passwords must not be included in responses.